Groq Tensor Streaming Processor 架构综述：软件定义硬件与确定性计算的范式革命

Claude (Anthropic)

2026 年 4 月

摘要

Groq Tensor Streaming Processor (TSP) 代表了一种根本不同的 AI 加速器设计哲学。本文基于 ISCA 2020/2022 两篇奠基性论文及 6 篇技术分析文章，系统综述了 Groq TSP 的架构创新。TSP 采用功能切片微架构，将 220MB SRAM 与矩阵/向量运算单元在空间上交错排列；通过软件定义硬件方法，将传统硬件中的动态调度、缓存管理、网络路由等复杂逻辑完全移至编译器静态处理；以确定性执行为核心，消除了缓存、仲裁器、分支预测器等所有非确定性元件，实现了从单芯片到万卡集群的完全可预测性能。

本文深入分析了 TSP 的 144 路 VLIW 指令集、320 元素向量处理单元、320×320 MACC 矩阵运算阵列、软件调度网络以及基于硬件对齐计数器 (HAC) 和软件对齐计数器 (SAC) 的多芯片同步机制。同时探讨了 Groq 被 NVIDIA 以 200 亿美元收购的技术背景，分析了 DSA 与 GPGPU 架构的融合趋势，并对 AI 加速器架构的未来演进提出了批判性思考。

关键词： Groq TSP；软件定义硬件；确定性执行；功能切片；张量流处理；AI 加速器；多芯片同步

目录

1 引言 **3**

1.1 背景与动机 3

1.2 核心创新点 3

1.3 论文组织 3

2 TSP 微架构设计 **4**

2.1 功能切片：重新组织处理器微架构 4

2.2 指令集架构：144 路 VLIW 6

2.3 存储层次：扁平化 SRAM 设计 7

2.4 各功能单元深入分析 8
2.4.1 指令控制单元 (ICU) 8
2.4.2 矩阵运算单元 (MXM) 8
2.4.3 向量运算单元 (VXM) 9
2.4.4 数据重排单元 (SXM) 9
2.4.5 流寄存器与数据传输机制 9

3 软件定义硬件与编译器架构 9
3.1 静态-动态接口划分 9
3.2 Groq 编译器架构 10
3.3 编译器优化技术详解 11
3.3.1 周期精确指令调度 11
3.3.2 模调度与软件流水线 12
3.3.3 寄存器压力感知调度 12
3.3.4 内存分配与数据布局优化 13
3.3.5 两阶段调度策略 13
3.4 运行时系统 13

4 多芯片同步与软件调度网络 13
4.1 从单芯片到多芯片的确定性扩展 13
4.2 软件调度网络 (SSN) 14

5 与 GPGPU 架构的对比分析 15
5.1 计算密度与能效 15
5.2 延迟与可预测性 16
5.3 编程模型与生态 16

6 NVIDIA 收购与技术整合展望 16
6.1 200 亿美元收购的技术逻辑 16
6.2 Rubin + Groq 3 LPU 的融合架构 17

7 结论与批判性思考 17
7.1 主要贡献总结 17
7.2 局限性与批评 17
7.3 未来研究方向 18
7.4 最终思考 18

1 引言

1.1 背景与动机

深度学习工作负载的爆炸式增长推动了专用加速器 (Domain-Specific Architecture, DSA) 的蓬勃发展。从 Google 的 TPU 到 NVIDIA 的 Tensor Core，业界一直在探索如何以更高效率执行神经网络计算。然而，这些设计大多遵循 GPGPU 的”硬件中心”范式：复杂的硬件调度逻辑、多层缓存层次结构、动态分支预测——这些机制虽然提高了平均性能，但也引入了不可预测的执行延迟和显著的面积/功耗开销。

Groq 公司于 2020 年在 ISCA 发表的 TSP 架构论文 [1] 提出了一个根本性的质疑：如果编译器能够在编译时精确知道程序的执行路径，为什么还要在硬件中保留复杂的动态调度逻辑？基于这一洞察，TSP 架构采用了一种”软件定义硬件”的设计理念——将控制和调度操作完全交由软件 (编译器) 完成，硬件仅负责执行。

这一设计理念的物理实现是一枚 14nm 工艺、25×29 mm、包含 268 亿晶体管的 ASIC 芯片 [1]。其核心规格令人印象深刻：220 MB 片上 SRAM 提供 55 TB/s 的 SRAM 带宽和 20 TB/s 的流寄存器带宽；4 个独立的 320×320 MACC 阵列提供 820 TeraOps/s 的峰值性能 (在 900MHz 频率下)；5,120 个 32 位向量 ALU 支持 FP16 和 INT8 原生数据类型；144 路 VLIW 指令发射支持多达 10,440 芯片的全局同步网络。

1.2 核心创新点

TSP 架构的革命性创新可以从三个层面理解。在芯片层面，TSP 采用功能切片架构，将内存 (MEM)、向量运算 (VXM)、矩阵运算 (MXM)、数据重排 (SXM) 四种功能单元在空间上交错排列，通过流寄存器实现确定性数据传输 [1]。这种设计消除了缓存、仲裁器、乱序执行等传统非确定性元件，使编译器能够进行周期精确的指令调度。

在编译器层面，TSP 将传统硬件控制逻辑 (指令调度、内存管理、数据流控制) 完全移至编译器。编译器精确知道每条指令的执行周期、每个数据的存储位置、每次通信的传输延迟。这种”全知”视角使编译器能够生成最优的指令序列，而硬件只需按序执行 [3]。

在系统层面，TSP 的多芯片扩展不依赖传统的动态路由网络，而是采用软件调度网络 (Software-Scheduled Networking, SSN)。所有网络路由决策在编译时确定，通过 Dragonfly 拓扑实现高带宽、低延迟的全局通信 [2]。硬件对齐计数器 (HAC) 和软件对齐计数器 (SAC) 机制确保数千个芯片能够保持纳秒级同步 [5]。

1.3 论文组织

本文剩余部分组织如下：第 2 节详细分析 TSP 的微架构设计，包括功能切片、指令集架构和存储层次；第 3 节探讨软件定义硬件的实现机制，重点分析编译器在指令调度和内存管理中的核心作用；第 4 节深入讨论多芯片同步机制，包括 HAC/SAC 和软

件调度网络；第 5 节将 TSP 与传统 GPGPU 架构进行对比分析；第 6 节探讨 Groq 被 NVIDIA 收购后的技术整合方向；第 7 节总结全文并提出批判性思考。

2 TSP 微架构设计

2.1 功能切片：重新组织处理器微架构

传统多核处理器采用”核心-缓存-互连”的层次化设计。每个核心包含私有 L1/L2 缓存，多个核心通过片上网络 (NoC) 共享 L3 缓存和主存。这种设计面临三个根本性问题：首先，深度学习工作负载的数据局部性难以被固定大小的缓存捕获，导致频繁的数据移动和缓存未命中；其次，缓存未命中、仲裁竞争、网络拥塞等因素导致执行时间难以预测；第三，乱序执行、推测执行、分支预测等机制消耗大量晶体管却仅带来边际性能提升。

Abts 等人 [1] 指出，在深度学习工作负载中，数据并行性极为丰富，可以天然映射到张量硬件。问题的关键不在于计算能力，而在于如何高效地将数据送入计算单元。基于这一观察，TSP 采用功能切片方法重新组织处理器微架构。图 1 展示了 TSP 的功能切片架构，芯片被划分为 20 个 SuperLane，每个 SuperLane 包含功能互补的四种单元。

图 1: TSP 功能切片架构图 [1]。图中显示了 MEM West/East、quad 组织方式、以及 MXM/VXM/SXM/C2C 等单元的空间布局。

表 1 总结了四种功能单元的职责分工：

图 2 展示了数据在这些功能单元间的流动方式。

每个 SuperLane 包含 16 个并行 lane，每个 lane 宽度为 320 元素。这种设计允许 TSP 以 SIMD(单指令多数据) 方式同时处理大量数据。关键创新在于流寄存器机制：流寄存器位于功能单元之间，作为生产者-消费者缓冲；数据从一个单元流出后立即被下一个单元消费，无需通过 SRAM 中转；编译器精确知道数据在流寄存器中的停留时间，可以进行精确调度。这种设计实现了极高的数据复用率。对于 ResNet50 推理，TSP 可以在

表 1: TSP 功能单元分类

单元类型	缩写	功能描述
内存单元	MEM	管理片上 SRAM，提供每个时钟周期 320 字的读写带宽
向量运算单元	VXM	执行元素级运算（激活函数、归一化、点积），每 lane 16 个 ALU
矩阵运算单元	MXM	执行矩阵乘法和卷积，每个 SuperLane 包含一个 320×16 MACC 阵列
数据重排单元	SXM	执行数据转置、广播、切片等重排操作

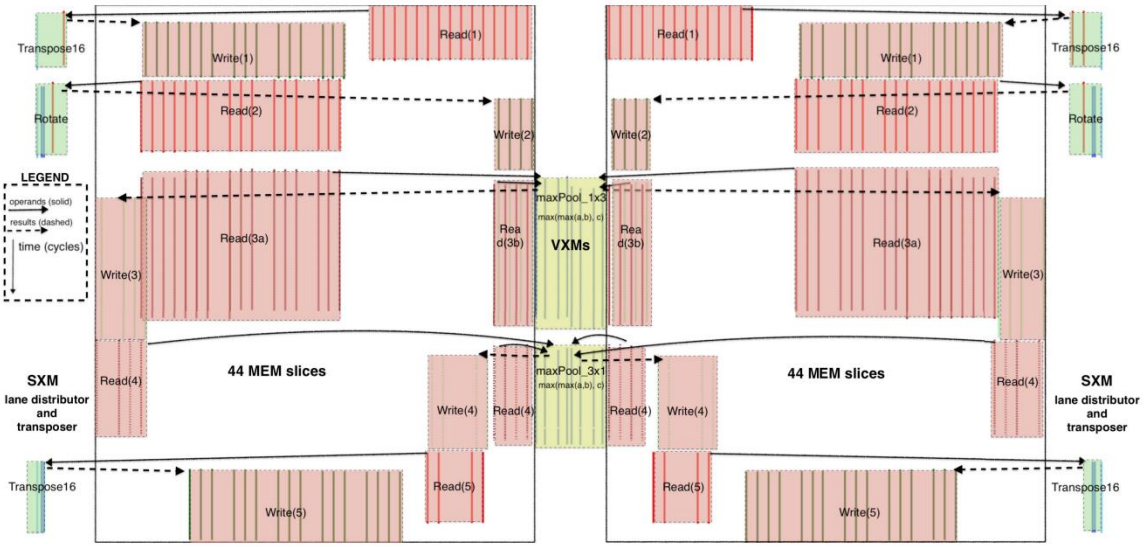

图 2: TSP 数据流与内存访问模式示意图 [1]。图中显示了 Read/Write 操作在 44 个 MEM slice 之间的流动，以及 VXM、SXM 等单元的交互。

batch size 为 1 的情况下达到 20.4K images/second 的处理速度，延迟仅 49 微秒 [1]——这比同期 GPU 快 4 倍以上。

2.2 指令集架构：144 路 VLIW

TSP 采用 144 路 VLIW(甚长指令字) 指令集。每条 VLIW 指令同时控制 20 个 SuperLane 与 5 个功能切片共 100 个独立控制信号，以及额外的同步、分支、网络控制信号。这种极端的 VLIW 设计之所以可行，是因为编译器拥有完整的信息：所有指令的执行延迟是固定的 (确定性设计)，所有数据的存储位置是已知的 (无缓存)，所有依赖关系可以在编译时解析 (无乱序执行)。

TSP 指令集包含多种关键指令类型 [3]，如代码 1 所示。

```
// 1. 基本运算指令
NOP       // 空操作，用于时序对齐
FMA       // 乘加运算，MXM核心操作
VADD/VMUL// 向量加减乘除

// 2. 内存访问指令
LOAD/STORE     // SRAM读写
PREFETCH       // 数据预取（软件控制）
GATHER/SCATTER // 非连续内存访问

// 3. 同步指令
SYNC          // 屏障同步
NOTIFY        // 事件通知
RUNTIME_DESKEW// 运行时去歪斜

// 4. 网络指令
SEND/RECV     // 芯片间数据发送/接收
MULTICAST     // 多播通信
REDUCE        // 归约操作

// 5. 控制指令
IFETCH        // 指令获取
BRANCH        // 条件分支（无推测）
SWITCH        // 上下文切换
```

Listing 1: TSP 核心指令类型

值得注意的是，TSP 的分支指令**不进行推测执行**。编译器在分支指令后插入足够多的 NOP，确保分支结果确定后才执行后续指令。这牺牲了部分性能换取了完全的可预测性。

2.3 存储层次：扁平化 SRAM 设计

TSP 最激进的设计决策之一是完全摒弃片外 DRAM。图 3 展示了 TSP 芯片的物理布局，整个 220 MB 存储由片上 SRAM 构成，分为 West Hemisphere 和 East Hemisphere 两个半区，各包含 44 个 slice。

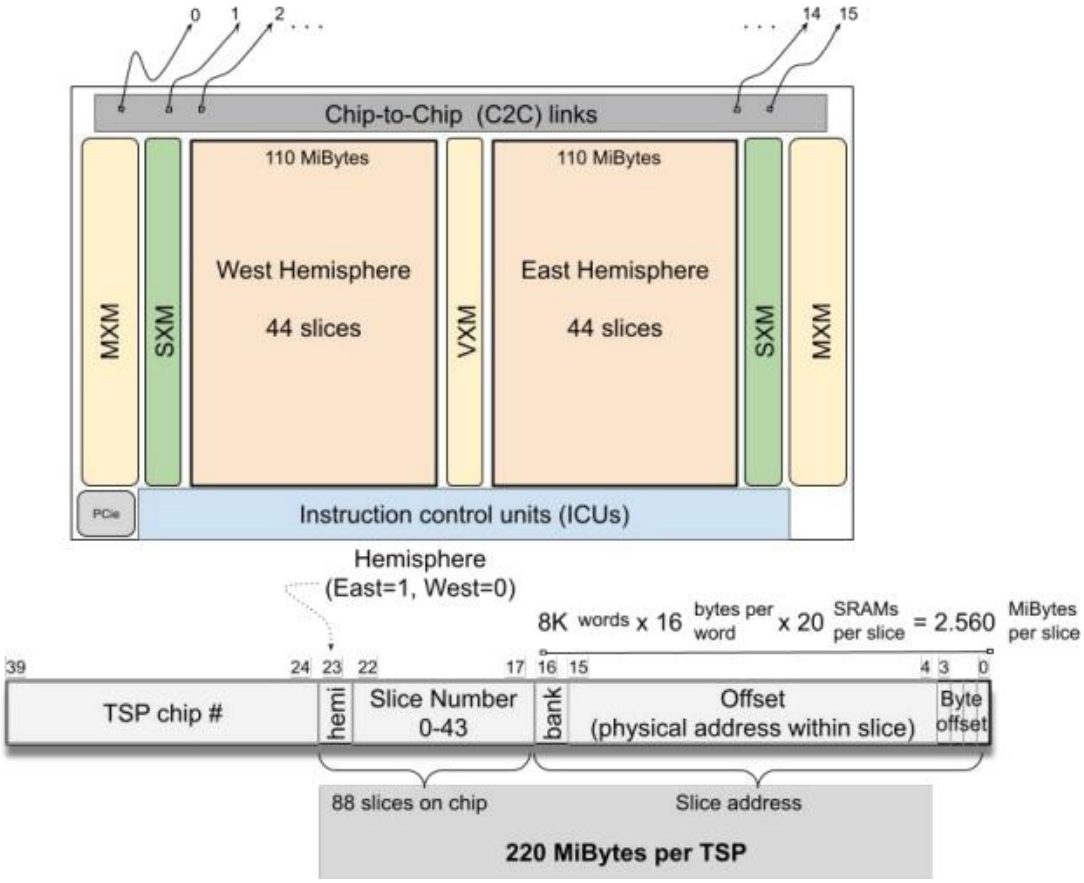

图 3: TSP 芯片物理布局与地址编码方案 [2]。West/East Hemisphere 各 44 slices，共 220 MiBytes，以及地址编码的详细格式。

具体而言，220 MB 存储分为两个 bank：Bank 0 存储权重，Bank 1 存储激活值。每个 bank 在每个时钟周期可以提供 320 字的读写带宽，总带宽达 55 TB/s。相比之下，GDDR6 显存的典型带宽仅为 1-2 TB/s。

由于没有缓存，TSP 要求编译器进行显式内存管理。程序员或编译器需要明确指定每个张量的 SRAM 存储位置、数据的生命周期和替换策略、内存 bank 之间的数据移动。这种显式管理增加了编译复杂度，但消除了缓存未命中的不可预测性。对于 ResNet50，TSP 编译器可以精确安排每层的权重加载和激活计算，实现零气泡流水线 [1]。

Groq 3 LPU 在存储子系统上进行了重大升级 [4]：SRAM 容量从 220 MB 提升至 500 MB，带宽从 80 TB/s 提升至 150 TB/s，新增 Fabric Expansion Logic (FEL) 支持通过 DDR 扩展内存。FEL 是一个 FPGA 芯片，通过高带宽 SerDes 连接 LPU 阵列，可以将 KV Cache 等大数据结构卸载到 DDR 内存，同时保持与片上 SRAM 类似的访问延迟特性。

2.4 各功能单元深入分析

基于对 HotChips 2024 演讲 [9] 和 OneFlow 技术文章 [7] 的深入分析，本节详细阐述 TSP 各功能单元的微架构设计和工作原理。

2.4.1 指令控制单元 (ICU)

TSP 包含 144 个指令控制单元 (Instruction Control Unit, ICU)，每个功能单元关联一个 ICU。ICU 的核心职责是从 SRAM 获取指令并控制关联功能单元的执行。所有 ICU 通过全局同步机制实现锁步 (lockstep) 执行，确保整个芯片的功能单元在同一周期执行对应指令。

ICU 支持的通用指令包括：IFETCH(从 SRAM 获取指令到 ICU 开始执行)、NOP(空操作，可重复 N 个周期用于时序对齐)、SYNC(将 ICU 置于等待状态，直到收到 NOTIFY 通知)、NOTIFY(唤醒其他 ICU 实现锁步执行)。这种设计使编译器能够精确推理每条指令的执行时间。

从面积开销角度，ICU 和所有指令调度逻辑仅占芯片面积的不到 3%[9]。相比之下，传统 CPU 中用于分支预测、乱序执行、推测执行的控制逻辑通常占用 15%-30%的芯片面积。这种极端简化的控制平面是 TSP 高计算密度的关键来源。

2.4.2 矩阵运算单元 (MXM)

MXM 是 TSP 架构的主力计算单元，每个 SuperLane 包含一个 320×16 的 MACC 阵列，整个芯片包含 4 个独立的 320×320 矩阵引擎。每个 MXM 平面存储 102,400 个权重，总计 409,600 个乘加器 (MACC)。在 900MHz 频率下，MXM 可提供约 750 TOPS(Int8Acc32) 的峰值算力，计算密度超过每平方毫米 1 TeraOps[9]。

MXM 支持的数据类型和运算模式如表 2 所示。

表 2: MXM 矩阵运算单元支持的数据类型与运算模式

数值模式	最大尺寸	每 MXM 密度	结果类型
INT8	$[N, 320] \times [320, 320]$	2 个/平面	INT32
FLOAT16	$[N, 320] \times [160, 320]$	1 个/平面	FLOAT32

MXM 的指令集包括：LW(Load Weights，从流加载权重到权重缓冲区)、IW(Install Weights，从流或 LW 缓冲区安装到 320×320 阵列)、ABC(Activation Buffer Control，协调激活值的到达)、ACC(Accumulate，累加 INT32 或 FP32 结果)。MXM 执行采用全流水线设计，每个周期可以接受新的输入，实现 sustained throughput。

2.4.3 向量运算单元 (VXM)

VXM 位于芯片的东西半球之间，包含 16 个处理元素 (PE) per lane，总计 5,120 个 32 位 ALU。每个 PE 每周期可执行一次 32 位运算或四次 INT8 运算。VXM 负责执行元素级运算，包括激活函数 (ReLU、GELU、Tanh)、归一化、点积等深度学习常见的逐点操作。

VXM 支持的指令类别包括：一元运算 (如 mask、negate)、二元运算 (如 add、mul、sub)、类型转换 (定点与浮点互转)、超越函数 (ReLU、TanH、Exp、RSqrt)。这种设计使 VXM 能够处理矩阵运算之外的复杂非线性变换，是端到端神经网络推理的关键。

2.4.4 数据重排单元 (SXM)

SXM 是 TSP 的" 瑞士军刀"，负责数据重塑、片上数据搬运和片外 I/O 操作。其核心功能包括：Shift up/down(在 lane 间上下移动数据 N 个位置)、Permute(320 输入到 320 输出的双射置换)、Distribute(在 SuperLane 内重排或复制数据)、Rotate(生成所有可能的旋转，支持 $n = 3$ 或 $n = 4$)、Transpose(16x16 矩阵转置)。

SXM 的重要性在于神经网络中频繁出现的数据重排操作：卷积的 im2col 转换、注意力机制的 transpose 操作、多播通信等。在传统 GPU 中，这些操作通常需要额外的内存访问，而 TSP 通过 SXM 在流寄存器之间直接完成，无需 SRAM 中转。

2.4.5 流寄存器与数据传输机制

流寄存器 (Stream Register) 是 TSP 功能切片架构的核心创新。功能单元之间的数据通过流寄存器传递，形成" 装配线" 式的数据流。关键特性包括：单周期延迟 (相邻功能单元间数据传输仅需 1 周期)、无仲裁 (编译器静态分配流寄存器，无需硬件仲裁)、确定性时序 (编译器精确计算数据到达时间)。

数据在芯片上东西向流动，充分利用空间局部性。编译器通过简单的减法即可计算数据移动时间：$T_{travel} = N_{hops} \times T_{cycle}$。这种确定性使编译器能够精确编排指令执行，无需硬件干预。

3 软件定义硬件与编译器架构

3.1 静态-动态接口划分

TSP 架构的核心理念是静态-动态接口划分。这一理念将计算系统的控制平面分为两部分：静态部分在编译时确定，包括指令调度、内存分配、数据布局、网络路由；动态部分在运行时确定，包括数据值、分支方向 (由编译器生成所有分支路径)、异常处理。传统架构将大部分控制逻辑放在硬件中动态执行，而 TSP 将静态部分完全移至编译器。这种划分的优势在于编译器拥有全局视角可以做出更优的调度决策，硬件简化节省的晶体管可用于更多计算单元，执行完全可预测便于验证和调试。

在 TSP 架构中，编译器扮演着核心角色：它知道每条指令将在哪个周期执行，知道每个数据位于哪个 SRAM 地址，知道每次芯片间通信的确切延迟，知道每个功能单元的状态。这种全知视角使编译器能够进行激进的优化：指令级并行、内存访问流水线、数据预取、循环展开等。更重要的是，这些优化在编译时验证，运行时无需任何猜测或回滚。

3.2 Groq 编译器架构

Groq 编译器 (groqit) 采用多阶段编译流程 [3]：前端接收 PyTorch/TensorFlow 模型并转换为中间表示 (IR)；图优化阶段执行算子融合、常量折叠、死代码消除；内存规划阶段为每个张量分配 SRAM 位置并规划生命周期；指令调度阶段生成 VLIW 指令序列并优化指令级并行；网络规划阶段生成芯片间通信的 SSN 路由表；后端生成 TSP 可执行的二进制代码。

内存分配是编译器面临的核心挑战之一。TSP 的 220 MB 或 500 MB SRAM 需要容纳：

- 模型权重（对于大模型可能需要分片存储在多个芯片）
- 激活值（中间计算结果）
- KV Cache（对于 LLM 推理）

编译器采用**内存池**（memory pool）和**张量生命周期分析**来优化内存使用，如代码 2 所示。

```python
def allocate_memory(tensors, max_size):
    # 按生命周期排序张量
    sorted_tensors = sort_by_lifetime(tensors)

    memory_map = {}
    for tensor in sorted_tensors:
        # 寻找合适的空闲块
        slot = find_free_slot(
            memory_map, tensor.size, tensor.lifetime)
        if slot is None:
            # 尝试与已分配的张量共享内存
            slot = find_sharing_opportunity(
                memory_map, tensor)
        if slot is None:
            raise OutOfMemoryError(
                f"Cannot allocate {tensor.name}")
        memory_map[tensor] = slot

    return memory_map
```

Listing 2: 内存分配伪代码

TSP 编译器的调度算法需要确保：

1. 每个功能单元在同一时间只执行一个操作
2. 数据依赖被正确满足（生产者先于消费者）
3. 内存访问不冲突（同一 SRAM bank 同时只服务一个请求）
4. 网络链路不冲突（软件调度网络的约束）

这本质上是**资源约束调度**问题，编译器使用启发式算法（如列表调度）结合整数线性规划（ILP）来求解最优调度。

3.3 编译器优化技术详解

TSP 编译器的核心挑战是在满足资源约束的前提下，生成周期精确的 VLIW 指令序列。基于近期 VLIW 编译器优化研究 [13][14] 和 Groq 技术资料 [9]，本节深入分析 TSP 编译器采用的关键优化技术。

3.3.1 周期精确指令调度

TSP 编译器执行**周期精确指令调度** (Cycle-Accurate Instruction Scheduling)，这是其实现确定性执行的基础。由于功能单元以锁步方式执行，编译器需要：确定每条指令的发出周期、计算操作数到达功能单元的精确时间、安排结果写回流寄存器的时序、处理功能单元间的流水线延迟。

这种调度可形式化为**资源约束的列表调度**问题：给定数据流图 $G = (V, E)$，其中节点 $v \in V$ 表示操作，边 $(u, v) \in E$ 表示数据依赖；给定资源约束 $R = \{r_1, r_2, ..., r_k\}$，每个资源有容量限制；目标是最小化调度长度 (makespan)，同时满足所有依赖和资源约束。

TSP 采用启发式列表调度算法：计算每个节点的优先级 (通常基于关键路径长度)，按优先级排序形成就绪列表，依次调度就绪节点到最早可用时隙，更新资源占用状态，重复直至所有节点调度完成。算法伪代码如代码 3 所示。

```
def list_schedule(dataflow_graph, resources):
    # 计算每个节点的关键路径长度作为优先级
    priorities = compute_critical_path(dataflow_graph)

    # 按优先级降序排序形成就绪列表
    ready_list = sort_by_priority(dataflow_graph, priorities)

    schedule = {}
    resource_state = initialize_resources(resources)

    while ready_list:
        node = ready_list.pop(0)

        # 计算最早开始时间(考虑数据依赖)
```

```
        earliest_time = compute_earliest_time(node, schedule)

        # 在最早时间后寻找满足资源约束的时隙
        for t in range(earliest_time, MAX_CYCLES):
            if check_resource_availability(node, t, resource_state):
                schedule[node] = t
                update_resources(node, t, resource_state)
                break

        # 将新就绪的节点加入列表
        ready_list.extend(get_newly_ready_nodes(dataflow_graph, node))
        ready_list.sort(key=lambda n: priorities[n])

    return schedule
```

Listing 3: 列表调度算法伪代码

3.3.2 模调度与软件流水线

对于循环密集型工作负载 (如 LLM 推理中的 token 生成)，TSP 编译器采用**模调度** (Modulo Scheduling)[14] 实现软件流水线。模调度的目标是找到最短的启动间隔 (Initiation Interval, II)，使得循环迭代可以流水线化执行而不发生资源冲突。

关键概念包括：启动间隔 II — 连续两次迭代开始之间的时间；最小 II(II_{min}) — 由资源约束或循环回边决定的理论下限；循环主体被划分为 II 个阶段，每个阶段形成一个**调度实体**。

模调度算法首先尝试 $II = II_{min}$，然后使用列表调度尝试将循环操作打包到 II 个阶段中。如果失败，则递增 II 重试。对于 TSP 的 VLIW 架构，成功的模调度可以实现每个周期启动一个新迭代，达到峰值吞吐量。

3.3.3 寄存器压力感知调度

传统列表调度仅关注指令级并行，可能产生大量同时活跃的值，导致寄存器溢出。TSP 编译器采用**寄存器压力感知调度** [15]，在并行性和寄存器使用之间取得平衡。

核心策略包括：在列表调度的优先级计算中引入寄存器压力估计；当寄存器压力接近阈值时，优先调度能释放寄存器的节点 (消费者)；使用图神经网络 (GNN) 预测调度决策对寄存器压力的影响；采用双寄存器技术 (Double Register Technique) 重叠寄存器加载与 MAC 运算。

研究表明 [15]，寄存器压力感知调度可以将因寄存器压力导致的不可调度程序比例从 18.6%降至 0.8%，对 97.5%的程序达到最优调度。

3.3.4 内存分配与数据布局优化

TSP 编译器的内存优化不仅包括简单的分配，还涉及复杂的数据布局决策：张量切片 (Tensor Tiling) — 将大张量划分为适合 SRAM 的小块；bank 冲突避免 — 通过地址映射确保并行访问不同 bank；权重预加载 — 在计算当前层时预加载下一层权重；激活值复用 — 最大化相邻层间的激活值重用。

这些优化可形式化为**二维约束动态规划**问题 [13]：在时间和空间两个维度上优化张量放置。相比传统启发式算法，该方法可达到接近最优的解，效率提升 58.34%。

3.3.5 两阶段调度策略

TSP 编译器采用 **Pre-RA** 和 **Post-RA** 两阶段调度 [13]：

1. **Pre-RA 调度** (寄存器分配前)：基于虚拟寄存器进行粗粒度调度，确定指令大致顺序，估计寄存器需求。
2. **Post-RA 调度** (寄存器分配后)：基于物理寄存器进行精调，插入 load/store 操作，优化内存访问流水线。

研究表明 [13]，Post-RA 调度相比仅 Pre-RA 调度平均可提供 14.04%的加速。这是因为寄存器分配后编译器知道确切的 spills 位置，可以更好地隐藏内存延迟。

3.4 运行时系统

与传统 GPU 需要复杂的运行时管理内核启动、内存分配、流同步不同，TSP 的运行时极为轻量：无需动态内存分配 (编译时已确定)，无需内核启动开销 (指令流连续执行)，无需流同步 (编译时已确保同步点)，无需动态调度 (硬件按序执行)。TSP 运行时主要负责加载编译后的程序到芯片、初始化 HAC/SAC 计数器、触发程序执行、处理异常和错误 (通过软件重放机制)。

TSP 架构通过多种机制确保可靠性 [2]：所有 SRAM 和网络链路都使用 ECC 保护，SerDes 链路使用前向纠错，检测到的错误触发软件重放重新执行出错的指令序列。由于执行的完全确定性，软件重放可以精确复现出错时的状态，大大简化了错误诊断和恢复。

4 多芯片同步与软件调度网络

4.1 从单芯片到多芯片的确定性扩展

单芯片的确定性执行相对容易实现——所有元件共享同一个时钟域，编译器可以精确计算每条指令的周期。但当系统扩展到多个芯片时，挑战出现：不同芯片的晶振频率

存在微小差异导致时钟漂移，芯片间链路的延迟可能因温度、电压变化而波动，传统的屏障同步会引入显著延迟。

Groq 的解决方案是分布式计数器同步机制，包括硬件对齐计数器 (HAC) 和软件对齐计数器 (SAC)。HAC 是一个 8 位计数器，每 256 个时钟周期回绕一次 [2]。其核心思想是：所有芯片的 HAC 在系统启动时通过测量链路延迟进行对齐，提供粗粒度的全局时间基准；编译器在代码中插入 HAC 同步点，确保所有芯片在同一 HAC 值时到达同步点。

系统启动时的对齐过程如算法 1 所示 [5]。该算法通过测量链路延迟，实现所有芯片 HAC 计数器的初始对齐。

Algorithm 1 HAC 初始对齐算法

Require: 所有芯片已上电并互联
Ensure: 所有芯片的 HAC 计数器同步
1: 选择一个芯片作为 Master
2: **for** 每个 Slave 芯片 **do**
3: Master 发送对齐请求
4: Slave 记录接收时间戳 t_1
5: Slave 发送响应
6: Master 记录接收时间戳 t_2
7: Master 计算往返延迟 $RTT = t_2 - t_1$
8: Master 计算单向延迟 $d = RTT/2$
9: Master 通知 Slave 其延迟偏移量 d
10: Slave 根据 d 调整 HAC 初始值
11: **end for**
12: 所有芯片开始同步计数

HAC 解决初始对齐问题，但长时间运行后芯片间的时钟漂移会导致同步失效。SAC 用于检测和补偿这种漂移：每个芯片维护一个本地 SAC 计数器，跟踪相对于全局时间的漂移量；当漂移超过阈值时，触发 RUNTIME_DESKEW 指令重新同步。RUNTIME_DESKEW 是 TSP 指令集中的关键同步指令，暂停执行等待所有芯片到达同步点，重新对齐计数器补偿累积的时钟漂移。编译器在程序的关键点 (如层边界、通信边界) 插入该指令，确保长期运行的稳定性。

4.2 软件调度网络 (SSN)

传统多芯片系统使用动态路由网络 (如 InfiniBand、NVLink)。动态路由的问题在于非确定性延迟 (路由决策依赖于网络状态)、拥塞 (热点流量导致网络拥塞，尾延迟显著)、复杂控制逻辑 (路由表、拥塞控制、流控机制消耗大量资源)。

Groq 的软件调度网络 (SSN) 采用完全不同的方法 [2]：所有路由决策在编译时确定，运行时无需路由计算；网络链路按时间片分配给不同流量；编译器确保同一链路不会同时被多个流量使用从而避免拥塞；采用 Virtual Cut-Through 技术使数据包直接穿过中间芯片无需缓冲。

TSP 使用 Dragonfly 拓扑进行多芯片互联。Dragonfly 拓扑的特点是高基数 (每个交换机连接多个节点)、短直径 (任意两个节点间最多 3 跳)、模块化 (芯片可以组织成组，组内全连接，组间通过全局链路连接)。在最大配置中，TSP 系统可以扩展到 10,440 个芯片，提供超过 2 TB 的全局内存，端到端延迟小于 3 微秒 [2]。

5 与 GPGPU 架构的对比分析

5.1 计算密度与能效

表 3 对比了 TSP 与同期 GPU 的关键指标。图 4 进一步从架构设计哲学层面展示了 TSP 与传统 CPU/GPU 的根本差异。

特性 (Feature)	**Groq TSP (张量流处理器)**	**传统架构 (CPU/GPU)**
调度方式	静态调度: 所有操作由编译器在编译时预先规划好.	动态调度: 由硬件调度器在运行时动态决定指令和数据流.
核心组件	仅包含执行单元和内存, 无动态控制逻辑. 硬件执行一个预先确定的"脚本".	包含复杂的反应式组件, 如缓存, 乱序执行引擎, 硬件调度器, 仲裁器等.
性能可变性	零方差: 每次运行的性能完全相同.	高方差: 性能受缓存命中率, 分支预测, "邻居"干扰等因素影响, 存在抖动.
长尾延迟	不存在: 架构设计从根本上消除了计算核心的长尾延迟.	普遍存在: 动态调度和资源争用是长尾延迟的主要来源.
编译器角色	核心决策者: 负责全部的执行规划, 资源分配和时间编排.	辅助角色: 主要负责将高级语言翻译成机器指令, 大量优化依赖于硬件在运行时动态完成.
调试与分析	简单: 静态性能报告在编译时即可生成, 无需在硬件上运行. 行为可重复, 易于调试.	复杂: 需要动态分析 (profiling) 工具来理解性能瓶颈, 结果可能因运行环境而异.

图 4: TSP 与传统架构 (CPU/GPU) 的核心差异对比 [3]。涵盖调度方式、核心组件、性能可变性、长尾延迟、编译器角色、调试与分析等六个维度。

从表中可以观察到，TSP 的计算密度 (Ops/sec/transistor) 是 V100 的 5 倍 [1]。这种优势主要来源于：晶体管用于计算而非控制逻辑 (无缓存、无乱序执行单元)，极高的片上内存带宽 (80 TB/s vs 2 TB/s)，以及确定性执行消除了性能抖动。

表 3: TSP 与 GPGPU 架构对比

指标	**TSP (2020)**	**V100**	**A100**
工艺节点	14nm	12nm	7nm
芯片面积	725 mm^2	815 mm^2	826 mm^2
晶体管数量	26.8B	21.1B	54.2B
峰值性能 (FP16)	820 TOPS	130 TFLOPS	312 TFLOPS
SRAM/缓存容量	220 MB	16 MB	40 MB
内存带宽	80 TB/s (SRAM)	900 GB/s (HBM2)	2 TB/s (HBM2e)
计算密度 (每晶体管性能)	30.6K	6.2K	5.8K
ResNet50 Batch=1	20.4K IPS	4-5K IPS	5-6K IPS

5.2 延迟与可预测性

对于 LLM 推理等延迟敏感型应用，TSP 表现出显著优势 [8]。单 token 延迟方面，TSP 可实现亚微秒级延迟，而 GPU 通常为毫秒级。尾延迟方面，TSP 的 P99 延迟几乎等于平均延迟，GPU 的 P99 延迟可能是平均值的 2-3 倍。抖动方面，TSP 无抖动，GPU 因缓存未命中、调度竞争等因素产生显著抖动。

确定性执行带来的不仅是性能优势，还包括：可以为关键任务提供硬实时保证，相同的输入总是产生相同的输出和时序从而简化调试，以及消除了时序侧信道攻击的可能性。

5.3 编程模型与生态

TSP 的软件定义方法带来了编程复杂度的权衡。优势在于编译器自动处理并行化、内存管理、同步，确定性执行简化了推理和调试，无需手动优化 (如 CUDA 中的共享内存管理、warp 调度)。劣势在于不支持动态形状，不支持动态内存分配 (如 paged attention)，需要重新编译模型以适配不同 batch size。

Groq 面临的最大挑战是生态系统 [3]：软件栈不成熟 (相比 CUDA 的成熟生态)，开源社区对 TSP 架构的了解和支持有限，专用硬件难以移植到其他平台存在供应商锁定风险。

6 NVIDIA 收购与技术整合展望

6.1 200 亿美元收购的技术逻辑

2024 年，NVIDIA 以 200 亿美元收购 Groq[6]。这一交易的技术逻辑在于：推理市场的竞争——随着 AI 推理需求爆发，低延迟、高吞吐的推理加速器变得至关重要；技术互补性——NVIDIA 的 GPU 在训练领域无敌，但在低延迟推理领域面临 Cerebras、SambaNova 等挑战；以及人才获取——Groq 团队拥有独特的确定性计算专业知识。

NVIDIA GPU 面临的根本问题是 [3]：内存墙 (HBM 带宽增长跟不上计算能力增长)，控制开销 (CUDA core 中约 30%的晶体管用于控制逻辑)，以及不可预测性 (缓存、分支预测、动态调度导致执行时间难以预测)。

6.2 Rubin + Groq 3 LPU 的融合架构

NVIDIA 发布的 Rubin GPU 与 Groq 3 LPU 的融合架构采用了 AFD(Attention-FFN 解耦) 设计 [4]。Rubin GPU 负责计算密集型任务 (Prefill 阶段的矩阵乘法)，Groq 3 LPU 负责内存带宽密集型任务 (Decoding 阶段的 FFN 计算)。这种异构设计利用了两类处理器的优势：GPU 的高计算密度和 LPU 的高内存带宽。

Groq 3 LPU 相比初代 TSP 有显著提升 [4]：SRAM 容量从 220 MB 提升至 500 MB，带宽从 80 TB/s 提升至 150 TB/s，具备 1.2 PFLOPS FP8 计算能力，配备 96 个 C2C 链路支持 112G SerDes。一个 LPX ComputeTray 包含 8 个 LPU，提供 9.6 PFLOPS 计算能力和 4 GB 聚合 SRAM。32 个 tray 组成一个机柜，总共 256 个 LPU 和 128 GB SRAM。

NVIDIA 可能从以下几个方面整合 Groq 技术 [3]：将 Groq 的确定性调度技术引入 CUDA 编译器，在 GPU 中增加更多 SRAM 减少 HBM 访问，将 SSN 技术应用于 NVLink 和 InfiniBand，利用 Groq 的 SRAM 设计经验优化未来 3D 堆叠 GPU。技术整合也面临挑战：如何在保持 CUDA 兼容的同时引入确定性执行，大容量 SRAM 的成本远高于 HBM，以及 TSP 的专用设计是否适用于除 LLM 以外的 AI 工作负载。

7 结论与批判性思考

7.1 主要贡献总结

Groq TSP 架构对计算机体系结构领域做出了重要贡献。软件定义硬件范式证明了通过编译器完全控制硬件的可行性，将传统硬件控制逻辑移至软件层。确定性计算架构展示了消除所有非确定性元件后可以实现完全可预测的高性能执行。功能切片微架构提出了与核心-缓存模型根本不同的组织方式，实现了极高的数据局部性和带宽利用率。大规模同步机制 HAC/SAC 解决了分布式系统中时钟同步的经典难题。软件调度网络证明了编译时路由可以替代动态路由，消除网络拥塞和尾延迟。

7.2 局限性与批评

尽管 TSP 架构具有许多创新之处，但也存在明显的局限性。在适用域方面，TSP 的设计高度针对深度学习推理优化，对其他类型的工作负载支持有限：不适合训练 (需要大量权重更新，SRAM 容量不足)，不适合动态图 (control flow 必须在编译时确定)，不适合稀疏计算 (缺乏高效的稀疏矩阵支持)。

SRAM 容量瓶颈是另一个关键问题。220 MB 或 500 MB 的 SRAM 容量严重限制了

可支持的模型规模 [5]：70B 参数的 LLM 需要 300-500 个芯片，576 卡集群的互联开销显著，无法使用 paged attention 等动态内存优化技术。

编译复杂度也不容忽视。确定性执行要求编译器解决 NP-hard 的调度问题，对于复杂模型，编译时间可能成为瓶颈。

7.3 未来研究方向

基于对 TSP 架构的分析，以下方向值得进一步研究。混合架构研究如何在 GPGPU 和 DSA 之间找到最佳平衡点，NVIDIA 的 Rubin+LPU 融合是一个尝试。动态确定性探索是否可以在保持确定性的同时支持有限的动态行为 (如动态 batching)。编译器-硬件协同设计进一步优化编译器算法以支持更大规模的模型和系统。可靠性机制方面，软件重放是确定性架构的独特优势，值得深入研究。建立确定性执行的行业标准有助于促进生态系统发展。

7.4 最终思考

Groq TSP 架构代表了一种根本不同的设计哲学。它挑战了”硬件必须足够通用以适应各种工作负载”的传统观念，证明了专用化在特定领域的巨大价值。200 亿美元的收购价格反映了业界对这一技术路线的认可。然而，技术成功不等于商业成功——TSP 能否在 NVIDIA 的框架内继续发展，能否在更广泛的 AI 工作负载中证明其价值，仍有待观察。

无论如何，TSP 架构为计算机体系结构社区提供了宝贵的思想实验：如果我们重新设计处理器，不受既有范式的约束，会做出什么不同的选择？这个问题本身就具有重要的启发意义。

参考文献

[1] Dennis Abts, Jonathan Ross, Jonathan Sparling, et al. Think Fast: A Tensor Streaming Processor (TSP) for Accelerating Deep Learning Workloads. In *Proceedings of the 47th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2020. DOI: 10.1109/isca45697.2020.00023

[2] Dennis Abts, Garrin Kimmell, Andrew Ling, et al. A Software-Defined Tensor Streaming Multiprocessor for Large-Scale Machine Learning. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA)*, pp. 567-580, 2022. DOI: 10.1145/3470496.3527405

[3] zartbot. Groq LPU 架构详解. Technical Report, 2025. Available in knowledge base.

[4] zartbot. 详细谈谈 Rubin + Groq 3 LPU 架构. Technical Report, 2026. Available in knowledge base.

[5] ljw. Groq 多芯片同步方案. Technical Analysis, 2025. Available in knowledge base.

[6] 看到新闻的时候. 谈谈那个被 NV 看上值 20B 的 Groq. Technical Analysis, 2024. Available in knowledge base.

[7] OneFlow. Groq：从头设计一个张量流式处理器架构. Technical Article, 2025. Available in knowledge base.

[8] 知返. Groq LPU 架构和 LLM 推理性能分析. Technical Analysis, 2024. Available in knowledge base.

[9] Dennis Abts. The Groq Tensor Streaming Processor: Software-defined Scale-out Architecture for Deep Learning. In *HotChips 34*, 2024. Available in knowledge base.

[10] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*, 6th Edition. Morgan Kaufmann, 2019.

[11] Norman P. Jouppi, Doe Hyun Yoon, George Kurian, et al. Ten Lessons From Three Generations Shaped Google's TPUv4i. In *Proceedings of the 48th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2021.

[12] NVIDIA. NVIDIA H100 Tensor Core GPU Architecture. Technical Report, 2024.

[13] Yuzhou Deng, Mei Wen, Dong Wang, et al. Optimizing VLIW Instruction Scheduling via a Two-Dimensional Constrained Dynamic Programming. In *ACM Transactions on Design Automation of Electronic Systems*, 2024. DOI: 10.1145/3643135

[14] Mei Wen, Yuzhou Deng, Dong Wang, et al. 分簇式 VLIW 密码专用处理器的编译器后端优化研究. In *Journal of Chinese Computer Systems*, Vol. 44, No. 10, 2023.

[15] Sofia, et al. A Graph Neural Network Approach to Improve List Scheduling Heuristics Under Register-Pressure. In *IEEE International Conference on Intelligent Decision Technologies*, Sofia, Bulgaria, 2024.

[16] Ye et al. CHARM: Composing Heterogeneous Accelerators for Matrix Multiply on Versal ACAP Architecture. In *ACM Transactions on Reconfigurable Technology and Systems*, 2024.

[17] SPECTRUM Team. Time-Predictable Software-Defined Architecture with SDF-Based Compiler Flow for 5G Baseband Processing. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2020.